

2η Εργαστηριακή Άσκηση
Γρηγόρης Δελατόλας (03119202)
Γεώργιος Κυριάκος (03119204)

Ασκηση 1.1

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <sys/types.h>
#include <sys/wait.h>

#include "proc-common.h"

#define SLEEP_PROC_SEC 10
#define SLEEP_TREE_SEC 3

/*
 * Create this process tree:
 * A--B---D
 *   |   ^
 *   |   C
 */
void fork_procs(void)
{
    /*
     * initial process is A.
     */

    change_pname("A");
    printf("A: Sleeping...\n");

    /* Create B */

    pid_t p_b;
    int status_b;
    p_b = fork();

    if(p_b < 0) {
        /* fork B failed */
        perror("Fork failed in B creation");
        exit(1);
    }

    else if(p_b == 0) {
        /* We are in node B! */
    }
}
```

```
/* We are in node B! */

change_pname("B");
printf("B: Starting...\n");

/* Create C */

pid_t p_d;
int status_d;

p_d = fork();

if(p_d < 0){

    /* fork D failed */
    perror("Fork failed in D creation");
    exit(1);

}

if(p_d == 0){

    /* We are in node D! */
    change_pname("D");
    printf("D: Starting...\n");

    printf("D: Sleeping...\n");
    sleep(12);

    printf("D: Awake!\n");

    printf("D: Exiting.\n");
    exit(13);

}

/* We are in node B right now */
printf("B: Waiting for D...\n");
p_d = wait(&status_d);
explain_wait_status(p_d, status_d);
exit(19);

}
```

```
else if(p_b > 0){

    /* We are in node A right now */

    /* Creation of C */
    pid_t p_c;
    int status_c;
    p_c = fork();

    if(p_c < 0){

        /* fork C failed */
        perror("Fork failed in C creation");
        exit(1);
    }

    if(p_c == 0){

        /* We are in node C */
        change_pname("C");
        printf("We are in C node!\n");

        printf("C: Sleeping...\n");
        sleep(12);

        printf("C: Awake!\n");

        printf("C: Exiting.\n");
        exit(17);
    }

    printf("Waiting for C\n");

    p_c = wait(&status_c);
    explain_wait_status(p_c, status_c);

    printf("Waiting for B\n");

    p_b = wait(&status_b);
    explain_wait_status(p_b, status_b);

    printf("A: Exiting...\n");
```

```

        exit(16);
    }

}

/*
 * The initial process forks the root of the process tree,
 * waits for the process tree to be completely created,
 * then takes a photo of it using show_pstree().
 *
 * How to wait for the process tree to be ready?
 * In ask2-{fork, tree}:
 *     wait for a few seconds, hope for the best.
 * In ask2-signals:
 *     use wait_for_ready_children() to wait until
 *     the first process raises SIGSTOP.
 */
int main(void)
{
    pid_t pid;
    int status;

    /* Fork root of process tree */
    pid = fork();
    if (pid < 0) {
        perror("main: fork");
        exit(1);
    }
    if (pid == 0) {
        printf("A_PID = %d \n", getpid());
        /* Child */
        fork_procs();
        exit(1);
    }
}

```

```

/*
 * Father
 */
/* for ask2-signals */
/* wait_for_ready_children(1); */

/* for ask2-{fork, tree} */
sleep(10);

/* Print the process tree root at pid */
show_pstree(pid);
printf("\n");
/*show_pstree(1); the nodes B,C and D become children of init (pid = 1) */
/* for ask2-signals */
/* kill(pid, SIGCONT); */

/* Wait for the root of the process tree to terminate */
pid = wait(&status);
explain_wait_status(pid, status);

return 0;
}

```

Έξοδος προγράμματος:

```

A PID = 15695
A: Sleeping...
B: Starting...
Waiting for C
We are in C node!
C: Sleeping...
B: Waiting for D...
D: Starting...
D: Sleeping...

A(15695) qwqB(15696) qqqD(15698)
      mqC(15697)

C: Awake!
C: Exiting.
D: Awake!
D: Exiting.
My PID = 15695: Child PID = 15697 terminated normally, exit status = 17
Waiting for B
My PID = 15696: Child PID = 15698 terminated normally, exit status = 13
My PID = 15695: Child PID = 15696 terminated normally, exit status = 19
A: Exiting...
My PID = 15694: Child PID = 15695 terminated normally, exit status = 16

```

Ερώτηση 1

Με αυτήν την εντολή τερματίζουμε πρόωρα την διεργασία A. Για τον λόγο αυτό εμφανίζεται το μήνυμα: "My PID = 15915: Child PID = 15916 was terminated by a signal, signo = 9".

Αυτό που συμβαίνει αμέσως μετά είναι ότι οι διεργασίες B, C γίνονται παιδιά της διεργασίας init , με pid =1.

Ερώτηση 2

Η συνάρτηση getpid() επιστρέφει το pid της τρέχουσας διεργασίας. Στην προκειμένη περίπτωση η τρέχουσα διεργασία είναι του πατέρα του A. Άρα εμφανίζεται το δέντρο με ρίζα τον πατέρα του A, ο οποίος έχει κι άλλα παιδιά-διεργασίες πέρα από το A.

```
A_PID = 15951
A: Sleeping...
B: Starting...
Waiting for C
We are in C node!
C: Sleeping...
B: Waiting for D...
D: Starting...
D: Sleeping...

ex2.1 (15950) qwqA (15951) qwqB (15952) qqgD (15954)
                x          mqC (15953)
                mqsh (15962) qqgptree (15963)

C: Awake!
C: Exiting.
D: Awake!
D: Exiting.
My PID = 15951: Child PID = 15953 terminated normally, exit status = 17
Waiting for B
My PID = 15952: Child PID = 15954 terminated normally, exit status = 13
My PID = 15951: Child PID = 15952 terminated normally, exit status = 19
A: Exiting...
My PID = 15950: Child PID = 15951 terminated normally, exit status = 16
```

Ερώτηση 3

Κάθε επεξεργαστής έχει την δυνατότητα να εκτελεί μόνο μια διεργασία σε μία δεδομένη χρονική στιγμή. Έτσι, εάν δεν υπάρχει όριο μπορεί κάποιος χρήστης να χρησιμοποιήσει όλους τους διαθέσιμους επεξεργαστές και έτσι να μην υπάρχει δικαιοσύνη.

Ασκηση 1.2

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include "tree.h"
#include "proc-common.h"

static void create_fork_tree(struct tree_node *root)
{
    printf("%s Starting... \n", root->name);
    pid_t pid_child;
    int i,j,status;
    change_pname(root->name);
    if (root->nr_children == 0) sleep(5);

    for(i=0; i < root->nr_children; i++) {

        pid_child = fork();
        if (pid_child < 0) {perror("fork"); exit(1);}

        else if (pid_child == 0) {

            create_fork_tree(root->children + i);
            exit(1);

        }
    }
    for (j=0; j < root->nr_children; j++) pid_child = wait(&status);
    printf("%s Exiting... \n", root->name);
}
```

```
int main(int argc, char *argv[]) {

    pid_t pid_root;
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <input_tree_file>\n\n", argv[0]);
        exit(1);
    }
    struct tree_node *root;
    root = get_tree_from_file(argv[1]);

    pid_root = fork();
    if (pid_root < 0) {perror("fork"); exit(1);}
    else if (pid_root == 0) {

        create_fork_tree(root);
        exit(1); // to vazo tora

    }
    else {

        sleep(1);
        show_pstree(pid_root);

    }

    return 0;
}
```


Βάζοντας ως είσοδο αυτό το αρχείο proc.tree

```
A
4
B
C
D
J

B
2
E
F

E
0

F
0

C
0

D
0

J
1
S

S
0
```

Η έξοδος είναι η αναμενόμενη:

```
A Starting...
B Starting...
C Starting...
D Starting...
E Starting...
J Starting...
F Starting...
S Starting...

A (16050) qwqB (16051) qwqE (16054)
      x      mqF (16056)
      tqC (16052)
      tqD (16053)
      mqJ (16055) qqqs (16057)

oslab74@os-node2:~/ex2/2.2$ C Exiting...
D Exiting...
E Exiting...
F Exiting...
B Exiting...
S Exiting...
J Exiting...
A Exiting...
```

Ερώτηση 1

Αρχικά, δεν μπορεί να εμφανιστεί μήνυμα έναρξης παιδιού πριν το μήνυμα έναρξης του πατέρα του. Επίσης, λόγω της sleep δεν μπορούμε να είμαστε βέβαιοι για την σειρά τερματισμού των διεργασιών, γι αυτό που είμαστε σίγουροι είναι ότι τα παιδιά τερματίζουν πριν τους γονείς, λόγω του wait που κάνουν οι γονείς.

Επειδή απλά περιμένουμε λίγο χρόνο με την εντολή `sleep`, δεν μπορούμε να είμαστε σίγουροι ότι οι διεργασίες θα τερματίσουν με την σειρά που θέλουμε. Είμαστε σίγουροι όμως ότι τα παιδιά τερματίζουν πριν τους γονείς οι οποίοι περιμένουν να ενημερωθούν για την κατάστασή τους με την `wait(&status)`.

Άσκηση 1.3

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include "tree.h"
#include "proc-common.h"

static void create_fork_tree(struct tree_node *root)
{
    pid_t pid_child, p, array[root->nr_children];
    int i,j,status;
    change_pname(root->name);
    // if (root->nr_children == 0) raise(SIGSTOP);

    for(i=0; i < root->nr_children; i++) {

        pid_child = fork();
        if (pid_child < 0){perror("fork"); exit(1);}

        else if (pid_child == 0){
            //array[i] = getpid();
            create_fork_tree(root->children + i);
            exit(2);
        }
        array[i] = pid_child;    // #arrays = #parrents
    }
    wait_for_ready_children(root->nr_children);
    raise(SIGSTOP);           // last stop is A

    for(i=0; i < root->nr_children; i++) {
        struct tree_node *help = root->children + i;
        printf("PID = %ld, name = %s is awake\n", (long)array[i], help->name);
        kill(array[i], SIGCONT);
        p = wait(&status);
        //p = waitpid(array[i], &status, WIFEXITED(status) | WUNTRACED);
        //explain_wait_status(p,status);

    }
    //raise(SIGCONT);
    exit(3);
}

int main(int argc, char *argv[]) {

    int status;
    pid_t p, pid_root;
```

```

pid_t p, pid_root;
if (argc != 2) {
    fprintf(stderr, "Usage: %s <input_tree_file>\n\n", argv[0]);
    exit(1);
}
struct tree_node *root;
root = get_tree_from_file(argv[1]);

pid_root = fork();
if (pid_root < 0) {perror("fork"); exit(1);}
else if (pid_root == 0){
    create_fork_tree(root);
}
else{
    wait_for_ready_children(1);
    show_pstree(pid_root);
    printf("PID = %ld, name = %s is awake\n", (long)getpid(), root->name);
    kill(pid_root, SIGCONT);
    p = wait(&status);
    //wait(&status);
    //explain_wait_status(pid_root, status);
}

return 0;

```

Πράγματι, εάν το δέντρο που κατασκευάζουμε στο proc-tree είναι της μορφής: A ↔ 2 ↔ B ↔ C ↔ B ↔ 1 ↔ D ↔ D ↔ 0 ↔ C ↔ 0 ↔ τότε έχουμε την έξοδο

```

My PID = 16121: Child PID = 16123 has been stopped by a signal, signo = 19
My PID = 16122: Child PID = 16124 has been stopped by a signal, signo = 19
My PID = 16121: Child PID = 16122 has been stopped by a signal, signo = 19
My PID = 16120: Child PID = 16121 has been stopped by a signal, signo = 19

A(16121) qqB(16122) qqD(16124)
      mqC(16123)

PID = 16120, name = A is awake
PID = 16122, name = B is awake
PID = 16124, name = D is awake
PID = 16123, name = C is awake

```

Βλέπουμε ότι τα μηνύματα ενεργοποίησης τυπώνονται κατά βάθος.

Ερώτηση 1

Αρχικά, σε πρακτικό επίπεδο με τα σήματα δεν χρειάζεται να σκεφτόμαστε τον κατάλληλο συνδυασμό χρόνων sleep ώστε να εκτυπώνεται το δέντρο.

Επίσης, με τα σήματα μπορούμε να ελέγξουμε τη σειρά ενεργοποίησης και τερματισμού των διεργασιών

Ερώτηση 2

Η `wait_for_ready_children()` εξασφαλίζει ότι πρώτα θα τερματίσουν όλα τα παιδιά και μετά ο γονιός. Χωρίς αυτήν δεν είναι σίγουρη η ορθή σειρά των διεργασιών (γεγονός που μπορεί να οδηγήσει σε απρόσμενα αποτελέσματα) καθώς υπάρχει πιθανότητα ένας γονιός να τερματίσει πριν από κάποιο παιδί του.

Άσκηση 1.4

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include "tree.h"
#include "proc-common.h"

#define READ_END 0
#define WRITE_END 1

int create_fork_tree(struct tree_node *root)
{
    pid_t pid_child, p, array[root->nr_children];
    int i,j,status;
    change_pname(root->name);
    // if (root->nr_children == 0) raise(SIGSTOP);
    if (root->nr_children == 0){

        printf("Terminated node %s \n", root->name);
        return (atoi(root->name));
    }
    else {
        int values[2];

        for(i=0; i < 2; i++) {

            printf("i = %d \n",i);
            int pfd[2];
            if (pipe(pfd) < 0){ perror("pipe"); exit(1); }

            pid_child = fork();

            if (pid_child < 0){perror("fork"); exit(1);}

            else if (pid_child == 0){

                close(pfd[READ_END]);
                int x = create_fork_tree(root->children + i);
                printf("x = %d \n", x);
                char str[16];
                sprintf(str, "%d", x); // int to string
                write(pfd[WRITE_END], str, sizeof(str));
            }
        }
    }
}
```

```

        printf("before exit \n");
        exit(1);
    }
    close(pfd[WRITE_END]);
    p = wait(&status);
    explain_wait_status(p, status);
    char value[16];
    read(pfd[READ_END], &value, sizeof(value));

    printf("%d READ : %s \n", i, value);
    values[i] = atoi(value);
}

printf("%d \n", sizeof(root->name)); //root name is an array
char new[3];
for(j=0; j<3; j++) new[j] = (root->name)[j];
printf("%d \n", sizeof(new));
printf("%c \n \n", new[1]);

if (new[1] == '+') {

    printf("+ Execution \n");
    int sum = values[0] + values[1];
    return sum;
    exit(1);
}
if (new[1] == '*') {

    printf("* Execution \n");
    int product = values[0] * values[1];
    return product;
    exit(1);
}

}
}

```

```

int main(int argc, char *argv[]) {

    int status, value;
    int fd[2];
    pid_t p, pid_root;
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <input_tree_file>\n\n", argv[0]);
        exit(1);
    }
    struct tree_node *root;
    root = get_tree_from_file(argv[1]);

    if (pipe(fd) < 0) {
        perror("pipe");
        exit(1);
    }
    pid_root = fork();

    if (pid_root < 0) {perror("fork"); exit(1);}
    else if (pid_root == 0){

        printf("%s \n", root->name);
        printf("Start fork \n");
        int x = create_fork_tree(root);
        printf("%d \n", x);
        exit(1);
    }

    return 0;
}

```


Σημείωση

Στο αρχείο `proc.tree` εισάγουμε τους τελεστές `+` και `*` μέσα σε `"` όπως και φαίνεται παρακάτω. Γι αυτό τον λόγο χρησιμοποιούμε τον πίνακα `new[3]`, στον οποίο πάντα `new[0] = ' + '` και `new[2] = ' * '`, ενώ το `new[1]` είναι είτε `+` είτε `*`.

Με αρχείο `proc.tree` το ακόλουθο:

```
'*'
2
'+
'+

'+
2
5
6

5
0

6
0

'+
2
'*
2

'*
2
2
1

2
0

1
0

2
0
```

Η έξοδος είναι η αναμενόμενη, δηλαδή 44

```

i = 0
i = 0
Terminated node 2
x = 2
before exit
My PID = 16199: Child PID = 16200 terminated normally, exit status = 1
0 READ : 2
i = 1
Terminated node 1
x = 1
before exit
My PID = 16199: Child PID = 16201 terminated normally, exit status = 1
1 READ : 1
16
3
*

* Execution
x = 2
before exit
My PID = 16198: Child PID = 16199 terminated normally, exit status = 1
0 READ : 2
i = 1
Terminated node 2
x = 2
before exit
My PID = 16198: Child PID = 16202 terminated normally, exit status = 1
1 READ : 2
16
3
+

+ Execution
x = 4
before exit
My PID = 16194: Child PID = 16198 terminated normally, exit status = 1
1 READ : 4
16
3
*

* Execution
44

```

Ερώτηση 1

Το πλήθος των σωληνώσεων είναι ίσο με το διπλάσιο πλήθος των γονικών διεργασιών συν ένα.

Αλγεβρικά, έχουμε: $\# \text{ Σωληνώσεων} = 2 * (\# \text{ Πατεράδων}) + 1$

Γίνεται για κάθε αριθμητικό τελεστή να χρησιμοποιηθεί μια σωλήνωση. Αυτό μπορεί να επιτευχθεί αν γράφει το πρώτο παιδί δεδομένα, τα διαβάζει ο πατέρας και έπειτα περιμένει τα δεδομένα από το δεύτερο παιδί. Αφού τα διαβάσει και αυτά συνεχίζει.

Ερώτηση 2

Ουσιαστικά, μπορούμε να εκτελέσουμε ενέργειες παράλληλα, οπότε η πράξη θα εκτελεστεί πιο γρήγορα από το να τις εκτελούσαμε σειριακά.